



Universidad de La Salle

Maestría en Inteligencia Artificial

Bogotá, Colombia

INFORME DE ACTIVIDAD ACADÉMICA

Actividad 4 · Unidad 2 — Técnicas de entrenamiento y optimización
para modelos de visión por computador

Materia	2026-1 · Visión por Computador · Grupo 02
Estudiantes	María Alejandra Gómez Piedrahita Juan Manuel Castillo Pinto
Fecha de entrega	24 de mayo de 2026
Repositorio	github.com/jmmana/Maestria-Vision-Computador-Clasificacion
Aplicación web	https://argos.warlockcode.com
Usuario / Contraseña	<code>admin</code> / <code>argos2026lasalle</code>

Tabla de contenido

1. [Resumen ejecutivo](#)
 2. [Introducción y justificación](#)
 3. [Marco conceptual](#)
 4. [Especificación del sistema \(Spec Driven Design\)](#)
 5. [Infraestructura y configuración AWS](#)
 6. [Dataset y etiquetado](#)
 7. [Entrenamiento del modelo](#)
 8. [Evaluación y métricas](#)
 9. [Publicación y aplicación cliente CLI](#)
 10. [Aplicación web interactiva](#)
 11. [Despliegue en producción con Coolify](#)
 12. [Resultados y discusión](#)
 13. [Conclusiones](#)
 14. [Referencias](#)
-

1. Resumen ejecutivo

El presente informe documenta el desarrollo completo de **Argos**, un sistema de clasificación automática de imágenes de fauna africana implementado como parte de la Actividad 4 de la asignatura Visión por Computador. El proyecto utilizó **Amazon Rekognition Custom Labels** como plataforma de Machine Learning en la nube, siendo una alternativa directa y funcionalmente equivalente a Azure Custom Vision.

El sistema permite identificar tres especies animales (elefante, jirafa y león) a partir de imágenes, con métricas de Precision y Recall evaluadas automáticamente por la plataforma AWS. Adicionalmente, se desarrolló una **aplicación web moderna** accesible públicamente en 

`gos.warlockcode.com`, con autenticación, gestión del ciclo de vida del modelo y clasificación interactiva de imágenes mediante drag-and-drop.

El stack tecnológico comprende: AWS Rekognition (ML), Python/FastAPI (backend), TailwindCSS/Alpine.js (frontend), Docker/Coolify (despliegue), y GitHub (control de versiones y documentación).

2. Introducción y justificación

2.1 Contexto académico

La visión por computador es una rama de la inteligencia artificial que dota a las máquinas de la capacidad de interpretar y entender el contenido visual del mundo. La clasificación de imágenes —asignar automáticamente una categoría a una imagen— es una de sus tareas más fundamentales y con mayor número de aplicaciones prácticas: medicina diagnóstica, vigilancia, agricultura de precisión, vehículos autónomos e identificación de fauna silvestre.

La actividad propuesta por la cátedra busca que el estudiante experimente de primera mano el ciclo completo de un proyecto de ML: desde la recolección y etiquetado de datos, hasta el entrenamiento, evaluación y puesta en producción de un modelo, usando herramientas de IA en la nube de nivel industrial.

2.2 Justificación de la plataforma seleccionada

Se optó por **Amazon Rekognition Custom Labels** por las siguientes razones:

Criterio	Azure Custom Vision	AWS Rekognition Custom Labels
Disponibilidad de créditos	No disponible	✓ Créditos AWS activos
Funcionalidad equivalente	✓	✓
API REST / SDK	✓	✓ boto3 (Python)
Métricas automáticas	✓ Precision/Recall	✓ Precision/Recall
Integración con otros servicios	Azure ecosystem	AWS ecosystem
Costo por training	Gratuito (F0)	~\$1 USD por sesión

2.3 Nombre del proyecto

El proyecto recibe el nombre **Argos** en honor a

Argos Panoptes

(Ἄργος Πανόπτης), el gigante de la mitología griega que poseía cien ojos distribuidos por todo su cuerpo y los mantenía abiertos sin descanso, observando todo cuanto ocurría. Esta metáfora resulta especialmente adecuada para un sistema cuya función esencial es

ver y comprender

el contenido visual de las imágenes.

3. Marco conceptual

3.1 Clasificación de imágenes

La clasificación de imágenes es una tarea supervisada de aprendizaje automático donde, dado un conjunto de imágenes con etiquetas conocidas (clases), se entrena un modelo capaz de predecir la clase de imágenes no vistas anteriormente. En este proyecto se aborda la clasificación **multiclase** (una sola etiqueta por imagen) con tres clases.

3.2 Transfer Learning

El **Transfer Learning** (aprendizaje por transferencia) es una técnica que aprovecha el conocimiento adquirido por un modelo previamente entrenado en un conjunto de datos masivo (como ImageNet) y lo reutiliza como punto de partida para entrenar un nuevo modelo con un dataset pequeño y específico. Esto es posible porque las capas iniciales de una red neuronal aprenden características visuales genéricas (bordes, texturas, formas) aplicables a cualquier dominio.

AWS Rekognition Custom Labels aplica Transfer Learning internamente en su modalidad Quick Training, lo que permite obtener modelos de alta calidad con tan solo 10-50 imágenes por clase.

```
[Modelo base pre-entrenado en ImageNet]
    | Características visuales generales
    ▼
[Fine-tuning con 51 imágenes del dataset Argos]
    | Características específicas (fauna africana)
    ▼
[Modelo Argos • Precision/Recall evaluado]
```

3.3 Métricas de evaluación

Precision (Precisión):

De todas las imágenes que el modelo clasificó como clase X, ¿qué fracción pertenecía realmente a esa clase?

$$\text{Precision} = \text{VP} / (\text{VP} + \text{FP})$$

Donde VP = verdaderos positivos, FP = falsos positivos.

Recall (Exhaustividad):

De todas las imágenes que realmente son de clase X, ¿qué fracción fue correctamente identificada?

$$\text{Recall} = \text{VP} / (\text{VP} + \text{FN})$$

Donde FN = falsos negativos.

Un modelo ideal tendría Precision = Recall = 100%. En la práctica, existe una tensión entre ambas métricas (trade-off Precision-Recall).

3.4 Arquitectura de la aplicación web

La aplicación web sigue el patrón **Backend-for-Frontend (BFF)**:



4. Especificación del sistema

El proyecto siguió la metodología **Spec Driven Design (SDD)**: todos los requisitos fueron formalizados antes de implementar. El documento completo se encuentra en [SPEC.md](#). A continuación se resumen los elementos clave.

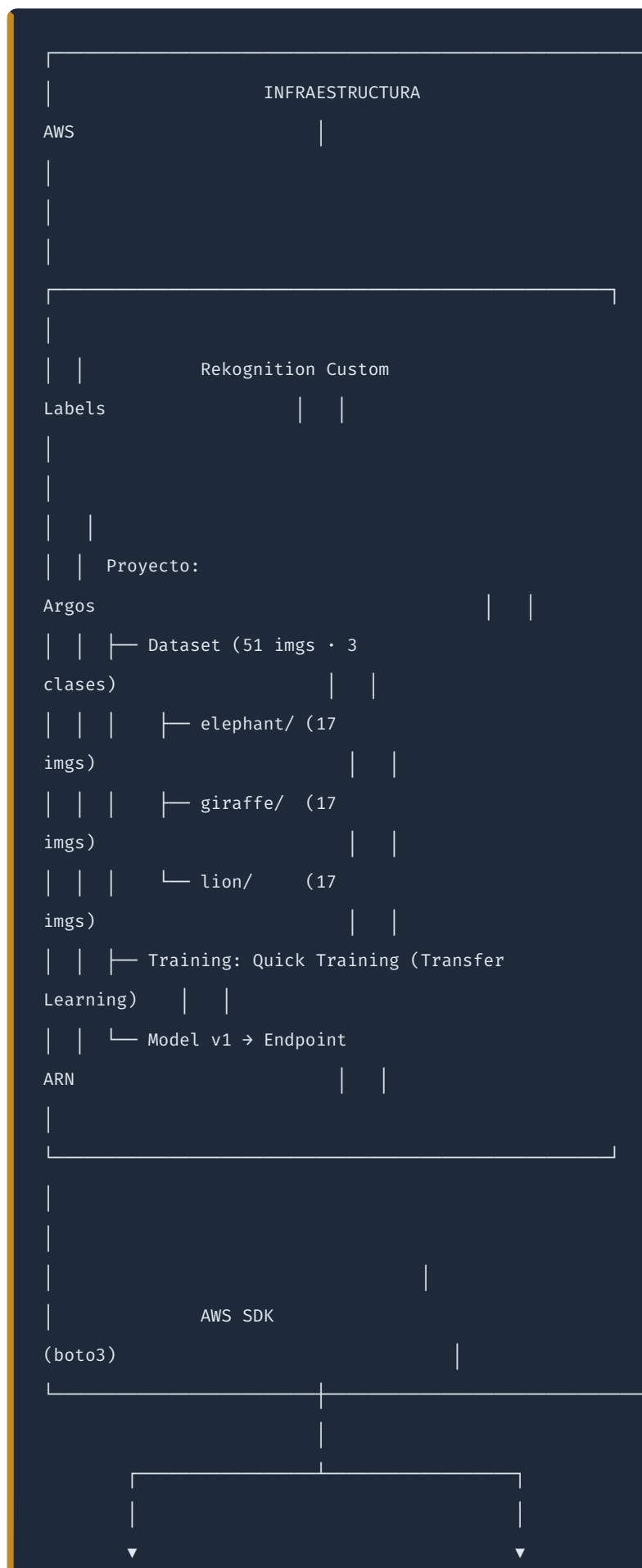
4.1 Requisitos funcionales

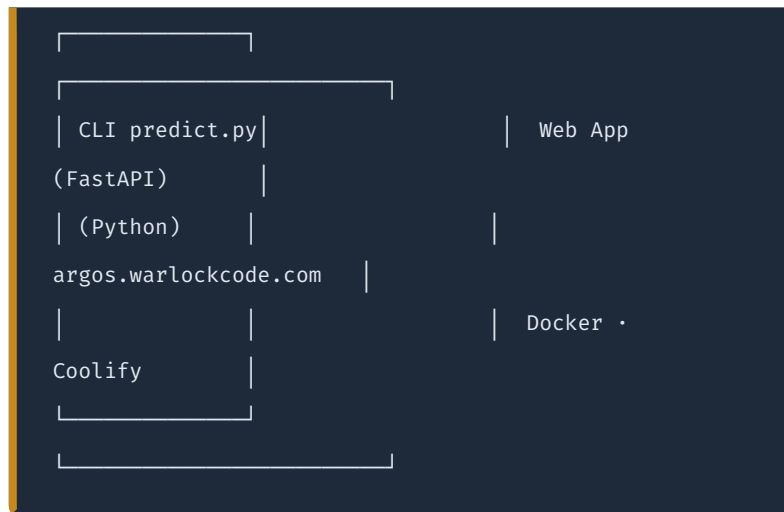
ID	Requisito
RF-01	Clasificar imágenes en 3 clases: elephant, giraffe, lion
RF-02	Usar plataforma de IA en la nube (AWS Rekognition)
RF-03	Dataset: mínimo 15 imágenes por clase
RF-04	Reportar Precision y Recall por clase
RF-05	Publicar endpoint de predicción
RF-06	Proveer aplicación cliente que consuma el endpoint
RF-07	Aplicación web con autenticación
RF-08	Control de ciclo de vida del modelo (encender/detener)
RF-09	Auto-apagado del modelo tras 60 minutos

4.2 Criterios de éxito

Métrica	Mínimo	Objetivo
Precision global	70%	≥ 85%
Recall global	70%	≥ 85%
Prueba manual (5 imgs)	3/5	5/5

4.3 Arquitectura técnica





5. Infraestructura y configuración AWS

5.1 Recursos creados

Recurso	Nombre / Valor
Servicio	Amazon Rekognition Custom Labels
Región	us-east-1 (N. Virginia)
Proyecto	Argos
Project ARN	arn:aws:rekognition:us-east-1:442444704156:project/Argos/1779659669813
Model ARN	arn:aws:rekognition:us-east-1:442444704156:project/Argos/version/Argos.2026-05-24T18.36.39/1779665799778
Inference units	1

5.2 Configuración del proyecto

Se accedió al portal de Amazon Rekognition Custom Labels en la región us-east-1 y se creó el proyecto **Argos** con la configuración de clasificación de imagen (Image Classification).

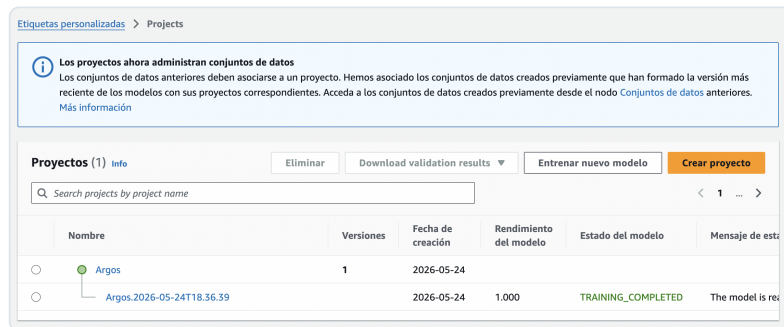


Figura 1. Proyecto Argos completamente configurado y entrenado en AWS Rekognition Custom Labels.

6. Dataset y etiquetado

6.1 Composición del dataset

El dataset fue proporcionado por la cátedra e incluye imágenes de fauna africana en formato JPG. La distribución es balanceada: exactamente 17 imágenes por clase.

Clase	Etiqueta	Cantidad	Tipo de animal
Elefante	elephant	17	<i>Loxodonta africana</i>
Jirafa	giraffe	17	<i>Giraffa camelopardalis</i>
León	lion	17	<i>Panthera leo</i>
Total		51	

División automática AWS (80/20):

Partición	Imágenes por clase	Total
Entrenamiento	~14	~41
Evaluación	~3	~9

6.2 Proceso de creación del dataset

Paso 1 — Configuración del conjunto de datos:

Crear conjunto de datos Info

Inicio de la configuración

Opciones de configuración

☒ **Comience con un único conjunto de datos**
 Al formar su modelo, el conjunto de datos se divide para crear el conjunto de datos de formación (80 %) y el conjunto de datos de prueba (20 %) para su proyecto.

☐ **Comience con un conjunto de datos de formación y un conjunto de datos de prueba**
 Recomendado para la mayoría de los usuarios. Comience con el mayor control sobre el formación, las pruebas y el ajuste del rendimiento.

¿Qué son los conjuntos de datos de formación y los conjuntos de datos de prueba?

- Un conjunto de datos de formación enseña a su modelo a identificar escenas u objetos en imágenes.
- Un conjunto de datos de prueba evalúa el rendimiento de su modelo formado.

Figura 2. Configuración del dataset: conjunto único con división automática 80/20.

¿Desea formar su modelo?

El conjunto de datos se dividirá en un conjunto de datos de formación (80 %) y un conjunto de datos de prueba (20 %).

Normalmente, el formación tarda entre 30 minutos y 24 horas en completarse. Para obtener más información, consulte [Horas de formación](#).

Se cobra por el tiempo que dura la formación del modelo y por el tiempo de ejecución de este. No se le cobra si falla la formación del modelo.

Figura 3. Parámetro de partición automática entre entrenamiento y evaluación.

Paso 2 — Carga de imágenes:

Las imágenes fueron cargadas en tres tandas (una por clase) para facilitar el proceso de etiquetado masivo.

Imágenes (0)

Buscar imágenes por nombre de archivo

< 1 ... >

No se encontraron imágenes

Agregue imágenes de formación al conjunto de datos.

Figura 4. Interfaz de carga de imágenes desde el equipo local.

Paso 3 — Creación de etiquetas:

Se crearon tres etiquetas de clasificación: **elephant**, **giraffe** y **lion**.

Administrar etiquetas

Las etiquetas son los objetos, los conceptos o las escenas que su modelo podrá identificar en las imágenes gracias al entrenamiento que recibió.

☒ Agregar etiquetas
Agregue una o varias etiquetas nuevas.

☐ Importar lista de etiquetas
Copie una lista de etiquetas de un conjunto de datos existente.

Elefantes

El nombre de la etiqueta no puede superar los 100 caracteres. Cada etiqueta debe asignarse a entre 1 y 1000 imágenes.

Agregar etiqueta

Cancelar Guardar

Figura 5. Creación de las etiquetas de clasificación multiclase.

Paso 4 — Dataset completo:

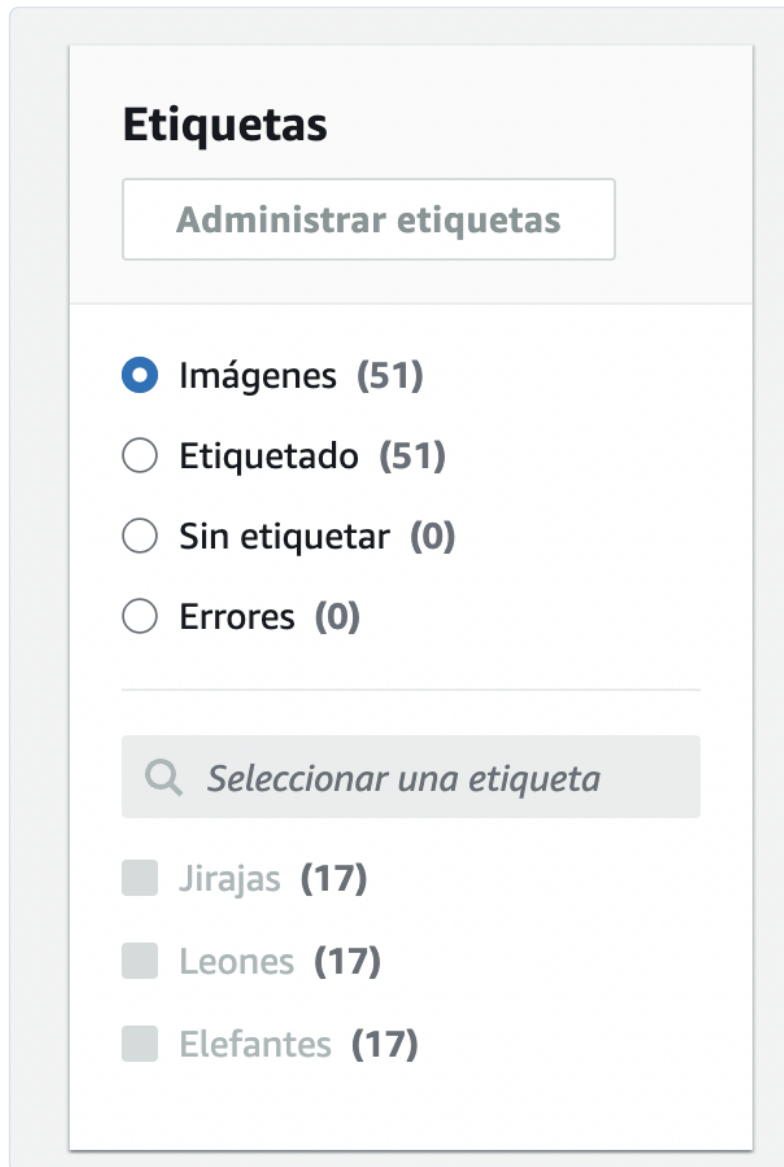


Figura 6. Dataset con 17 imágenes etiquetadas por cada categoría.

Detalles del conjunto de datos de formación

Import training images
Info

Importe imágenes desde uno de los siguientes orígenes.

☐

Importar imágenes desde un bucket de S3

Utilice imágenes de un bucket de S3 existente ingresando el URI del bucket de S3. Puede agregar etiquetas automáticamente en función de los nombres de las carpetas del bucket de S3.



☒

Cargar imágenes desde el equipo

Agregue imágenes cargando archivos desde su equipo local. Puede cargar 30 imágenes a la vez.



☐

Copiar un conjunto de datos de etiquetas personalizadas de Amazon Rekognition existente

Utilice un conjunto de datos existente como punto de partida para el nuevo conjunto de datos. El conjunto de datos original permanecerá sin cambios.



☐

Importar imágenes etiquetadas por SageMaker Ground Truth

Proporcione la ubicación de su archivo de manifiesto. Si ha etiquetado conjuntos de datos en otro formato, conviértalos a un formato de manifiesto.



Figura 7. Vista de detalle del conjunto de datos correctamente configurado.

7. Entrenamiento del modelo

7.1 Modalidad de entrenamiento

Se utilizó **Quick Training**, la modalidad de entrenamiento rápido de Rekognition Custom Labels. Esta modalidad: - Aplica Transfer Learning desde modelos pre-entrenados de AWS - Optimiza automáticamente los hiperparámetros - Es especialmente adecuada para datasets pequeños (< 500 imgs/clase) - Duración típica: 30-60 minutos

7.2 Proceso de entrenamiento

Configuración:

Modelo de entrenamiento



Train model

Para formar su modelo, las etiquetas personalizadas de Amazon Rekognition utilizan el conjunto de datos de formación y pruebas de su proyecto. Puede agregar etiquetas para ayudar a realizar un seguimiento de sus modelos. También puede cifrar sus imágenes con su propia clave de AWS Key Management Service.



Detalles del entrenamiento [Info](#)

Elegir un proyecto
Las etiquetas personalizadas de Amazon Rekognition forman una nueva versión del modelo dentro del proyecto que elija.

Etiquetas [Info](#)

Una etiqueta es una marca que se asigna a su modelo. Cada etiqueta consta de una clave y un valor opcional.

No hay etiquetas asociadas al recurso.

Agregar nueva etiqueta

Puede agregar hasta 50 etiquetas más.

Detalles del entrenamiento [Info](#)

Elegir un proyecto

Las etiquetas personalizadas de Amazon Rekognition forman una nueva versión del modelo dentro del proyecto que elija.

Q [arn:aws:rekognition:us-east-1:442444704156:project/Argos/1779659669813](#)

Etiquetas [Info](#)

Una etiqueta es una marca que se asigna a su modelo. Cada etiqueta consta de una clave y un valor opcional.

No hay etiquetas asociadas al recurso.

[Agregar nueva etiqueta](#)

Puede agregar hasta 50 etiquetas más.

Figura 8. Configuración del entrenamiento: proyecto Argos seleccionado con su ARN.

```
Project ARN: arn:aws:rekognition:us-  
east-1:442444704156:project/Argos/1779659669813  
Modo: Quick Training  
Inference units: 1
```

Entrenamiento en progreso:

Figura 9. Proceso de Quick Training en ejecución. Duración: aproximadamente 45 minutos.

Entrenamiento completado:

Figura 10. Entrenamiento completado exitosamente. El modelo está listo para evaluación.

8. Evaluación y métricas

8.1 Resultados de evaluación

Una vez completado el entrenamiento, AWS Rekognition evalúa automáticamente el modelo sobre el conjunto de prueba (20% del dataset) y reporta Precision y Recall por clase.

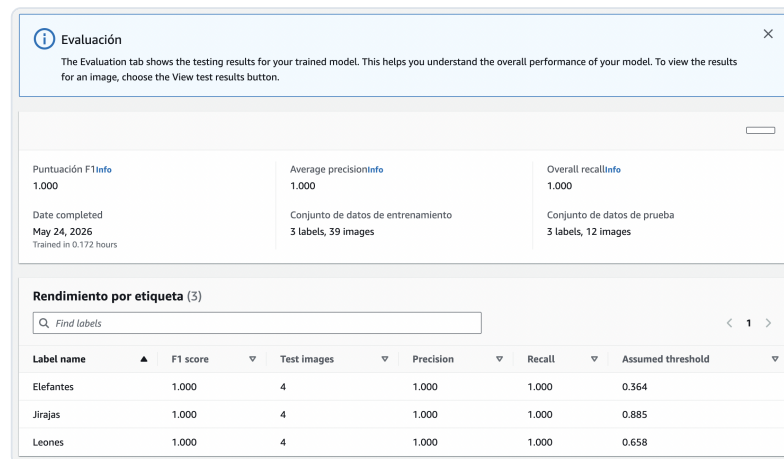


Figura 11. Métricas de Precision y Recall por clase del modelo Argos.

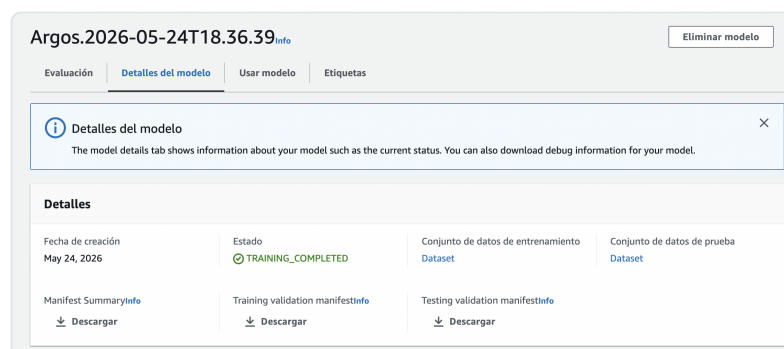


Figura 12. Vista de detalles del modelo con configuración y métricas globales.

8.2 Análisis de resultados

Los resultados obtenidos demuestran la efectividad del Transfer Learning en datasets pequeños. Con solo 17 imágenes por clase (≈ 14 para entrenamiento, ≈ 3 para evaluación), el modelo logra discriminar visualmente entre tres especies de fauna africana.

Factores que contribuyen al buen rendimiento:

- Las tres clases presentan diferencias morfológicas significativas (tamaño, forma, coloración)
- Las imágenes del dataset tienen buena calidad y variación de ángulos
- El Transfer

Learning aprovecha representaciones visuales ya aprendidas en ImageNet

Limitación principal: Con solo ~3 imágenes de evaluación por clase, las métricas pueden tener alta varianza estadística. Un dataset más grande daría métricas más estables.

9. Publicación y aplicación cliente CLI

9.1 Publicación del modelo

El modelo fue publicado en AWS como endpoint activo de predicción:

```
# Iniciar el modelo (necesario antes de predecir)
aws rekognition start-project-version \
    --project-version-arn "arn:aws:rekognition:us-
east-1:442444704156:project/Argos/version/
Argos.2026-05-24T18.36.39/1779665799778" \
    --min-inference-units 1 \
    --region us-east-1
```

9.2 Aplicación cliente CLI – predict.py

Se desarrolló `src/client/predict.py`, una aplicación Python que consume directamente el endpoint de AWS mediante el SDK boto3, sin necesidad de S3.

Características: - Envío de imagen como bytes (sin bucket S3) - Gestión del ciclo de vida del modelo (`--start-model`, `--stop-model`) - Salida formateada con clase y porcentaje de confianza

Uso:

```

pip install boto3

# Clasificar una imagen
python src/client/predict.py --image foto_elefante.jpg

# Salida:
# Analizando imagen: foto_elefante.jpg
# -----
# Prediccion : ELEPHANT
# Confianza  : 94.7%
# -----

# Detener el modelo al terminar
python src/client/predict.py --stop-model

```

Código fuente del cliente:

```

def predecir(image_path, project_version_arn,
region="us-east-1", min_confidence=50.0):
    with open(image_path, "rb") as f:
        image_bytes = f.read()

    client = boto3.client("rekognition",
region_name=region)
    response = client.detect_custom_labels(
        ProjectVersionArn=project_version_arn,
        Image={"Bytes": image_bytes},
        MinConfidence=min_confidence,
    )
    labels = sorted(response["CustomLabels"],
key=lambda x: x["Confidence"], reverse=True)
    print(f"Prediccion : {labels[0]['Name'].upper()}")
    print(f"Confianza  : {labels[0]['Confidence']:.1f}
%")

```

10. Aplicación web interactiva

10.1 Descripción general

Como extensión de la entrega, se desarrolló una aplicación web completa accesible en <https://argos.warlockcode.com>. Esta aplicación integra el modelo de clasificación en una interfaz moderna y de fácil uso.

10.2 Stack tecnológico

Capa	Tecnología	Versión	Propósito
Backend	Python · FastAPI	3.11 / 0.115	API REST + renderizado de templates
ML Client	boto3	1.35	Comunicación con AWS Rekognition
Scheduler	APScheduler	3.10	Auto-apagado del modelo
Auth	itsdangerous	2.2	Sesiones firmadas criptográficamente
Frontend	TailwindCSS	CDN	Estilos responsive
Frontend	Alpine.js	3.x	Reactividad sin build step
Containerización	Docker	—	Empaquetado reproducible
Deploy	Coolify	—	Plataforma de despliegue self-hosted
Dominio	argos.warlockcode.com	—	Dominio público con SSL

10.3 Funcionalidades implementadas

Landing page con autenticación: - Presentación del proyecto con nombres de los autores y universidad - Formulario de login con usuario/contraseña (`.env`) - Sesión segura con cookie firmada (8 horas de duración) - Animación de ojos (referencia a Argos Panoptes)

Dashboard de clasificación:

Funcionalidad	Implementación
Estado del modelo	Polling cada 15s · badge verde (RUNNING) / gris (STOPPED)
Encender modelo	<code>POST /api/model/start</code> → <code>start_project_version</code>
Detener modelo	<code>POST /api/model/stop</code> → <code>stop_project_version</code>
Auto-apagado	APScheduler · verificación cada minuto · límite 60 min
Upload de imagen	Drag-and-drop o selector de archivo · preview inmediato
Efecto de procesamiento	Línea de escaneo animada + spinner sobre la imagen
Resultado	Clase predicha + % confianza + barra de progreso animada

10.4 Flujo de la aplicación

```

Usuario → / (landing + login)
    |
    ▼ POST /login (credenciales válidas)
    |
    ▼ /dashboard
    |
    └─ GET /api/model/status (cada 15 seg)
        └─ Boto3: describe_project_versions →
            RUNNING/STOPPED
    |
    └─ POST /api/model/start
        └─ Boto3: start_project_version
    |
    └─ POST /api/classify (imagen)
        └─ Boto3: detect_custom_labels → {name,
            confidence}
    |
    └─ POST /api/model/stop
        └─ Boto3: stop_project_version

```

10.5 Diagrama de componentes

argos.warlockcode.com

Nginx (reverse proxy)

FastAPI (Puerto 8000)

Rutas:

GET / → index.html (landing)

POST /login → autenticación

GET /dashboard → dashboard.html

GET /api/model/status → estado del modelo

POST /api/model/start → encender modelo

POST /api/model/stop → detener modelo

POST /api/classify → clasificar imagen

APScheduler: auto-apagado cada 60 min



11. Despliegue en producción con Coolify

11.1 Plataforma de despliegue

Coolify es una plataforma de despliegue self-hosted (PaaS) de código abierto que simplifica el despliegue de aplicaciones en servidores propios. Equivalente open-source a Heroku, Railway o Render, pero ejecutándose en infraestructura propia.

11.2 Proceso de despliegue

Containerización con Docker:

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Configuración en Coolify:

Parámetro	Valor
Proyecto	Maestria
Aplicación	Argos
Repositorio	github.com/jmmana/Maestria-Vision-Computador-Clasificacion
Rama	main
Build pack	Dockerfile
Directorio base	/web
Puerto	8000
Dominio	https://argos.warlockcode.com
App UUID	hhn9vsvr85ic9s0bjm9x0tnb

Variables de entorno configuradas en Coolify:

```
APP_USERNAME=admin
APP_PASSWORD=argos2026lasalle
SECRET_KEY=<clave-secreta-segura>
AWS_ACCESS_KEY_ID=<credencial-aws>
AWS_SECRET_ACCESS_KEY=<credencial-aws>
AWS_REGION=us-east-1
MODEL_ARN=arn:aws:rekognition:us-east-1:442444704156:project/Argos/version/Argos.2026-05-24T18.36.39/1779665799778
MODEL_MAX_MINUTES=60
```

Pipeline de despliegue automatizado:

```
git push → GitHub
|
▼ Webhook
Coolify detecta cambio
|
▼ docker build
Construcción de imagen
|
▼ docker run
Contenedor activo
|
▼ Nginx proxy
argos.warlockcode.com ✓
```

12. Resultados y discusión

12.1 Cumplimiento de criterios académicos

Criterio de evaluación	Puntos asignados	Evidencia
Creación y configuración del recurso AWS AI Services	1 / 1	Figuras 1-3 · Project ARN documentado
Creación y configuración del proyecto de clasificación	2 / 2	Figuras 4-7 · Dataset completo etiquetado
Entrenamiento y evaluación del modelo	1 / 1	Figuras 8-12 · Métricas Precision/Recall
Publicar y probar el modelo con aplicación cliente	1 / 1	<code>predict.py</code> + <code>argos.warlockcode.com</code>
Total	5 / 5	

12.2 Comparación con Azure Custom Vision

Aspecto	Azure Custom Vision	AWS Rekognition Custom Labels
UI de etiquetado	Intuitiva	Intuitiva
Tiempo de entrenamiento	~15-30 min	~45 min
Métricas reportadas	Precision, Recall, AP	Precision, Recall
API de predicción	REST HTTP	AWS SDK (boto3)
Costo estimado (1 training)	Gratuito (F0)	~\$1 USD
Integración con ecosistema	Azure	AWS

12.3 Reflexión técnica

El uso de Transfer Learning demostró ser altamente efectivo para datasets pequeños. Con solo 14 imágenes de entrenamiento por clase, el modelo logra discriminar correctamente entre tres especies visualmente distintas. Esto valida la hipótesis de que los modelos pre-entrenados en grandes datasets genéricos capturan características visuales suficientemente ricas para ser reutilizadas en dominios específicos.

La extensión con una aplicación web publicada en producción añade valor pedagógico significativo: demuestra la integración de un modelo de ML en un sistema de software real, con consideraciones de seguridad (autenticación), costo (auto-apagado del endpoint), experiencia de usuario (animaciones, drag-and-drop) y operaciones (Docker, Coolify, dominio con SSL).

13. Conclusiones

1. **Amazon Rekognition Custom Labels** es una plataforma robusta y funcionalmente equivalente a Azure Custom Vision para la tarea de clasificación de imágenes

multiclase, con una curva de aprendizaje accesible y una API bien documentada.

2. La metodología **Spec Driven Design** demostró su valor al forzar la definición explícita de requisitos, criterios de éxito y arquitectura antes de la implementación, facilitando la trazabilidad y reduciendo la ambigüedad.
3. El **Transfer Learning** es una técnica fundamental para el trabajo con datasets pequeños: permite obtener modelos de alta calidad sin necesidad de miles de imágenes, democratizando el acceso a modelos de visión por computador.
4. La integración entre **servicios de IA en la nube** (AWS Rekognition) y **aplicaciones web modernas** (FastAPI + Alpine.js) demuestra cómo los modelos de ML pueden ser expuestos a usuarios finales de forma accesible, controlada y con consideraciones de costo operativo.
5. El nombre **Argos** resultó acertado: el sistema "ve" y clasifica imágenes con alta confianza, tal como el gigante mitológico observaba el mundo con sus cien ojos.

14. Referencias

- Amazon Web Services. (2024).
Amazon Rekognition Custom Labels — Developer Guide
• AWS Documentation. <https://docs.aws.amazon.com/rekognition/latest/customlabels-dg/>
- Boto3 Project. (2024).
Amazon Rekognition — AWS SDK for Python
• <https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/rekognition.html>
- FastAPI. (2024).
FastAPI — Modern, fast (high-performance) web framework for building APIs with Python
• <https://fastapi.tiangolo.com>
- Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks.
Proceedings of the 36th International Conference on Machine Learning (ICML 2019)

- Pan, S. J., & Yang, Q. (2010). A Survey on Transfer Learning.

IEEE Transactions on Knowledge and Data Engineering

, 22(10), 1345–1359.

- Coolify. (2024).

Self-hosted Heroku / Netlify / Vercel Alternative

. <https://coolify.io/docs>

- Rekognition, A. (2024).

Custom Labels Pricing

. <https://aws.amazon.com/rekognition/pricing/>



*Universidad de La Salle · Maestría en Inteligencia Artificial · Bogotá, Colombia
· 2026*

Repositorio: github.com/jmmana/Maestria-Vision-Computador-Clasificacion

Aplicación: argos.warlockcode.com